Лабораторна робота №3

PCA, K-means, Autoencoder,

RandomOverSample, SMOTE, ADASYN

Аналіз ефективності навчання автокодувальника

та порівняння класифікаторів з/без аугментації даних

Автор: Павло Шпагін

Платформа: Windows 10 (build 10.0.19045)

TensorFlow: 2.21.0

Python: 3.10.0

1. Системна інформація

Для виконання лабораторної роботи використовувалася наступна конфігурація:

```
Операційна система: Windows 10 (build 10.0.19045)
TensorFlow: 2.21.0
Python: 3.10.0
Процесор: Intel
Пристрої: [PhysicalDevice(name='/physical_device:CPU:0', device_type='CPU')]
```

Комп'ютер працює на процесорі Intel, без виділеної відеокарти (використовується лише CPU). Версія TensorFlow 2.21.0 (tensorflow-cpu).

2. Завантаження та підготовка даних

Дані були завантажені з файлу shaped.pickle. Розмірність вхідних даних: (297, 1200) (297 зразків, 1200 ознак кожен).

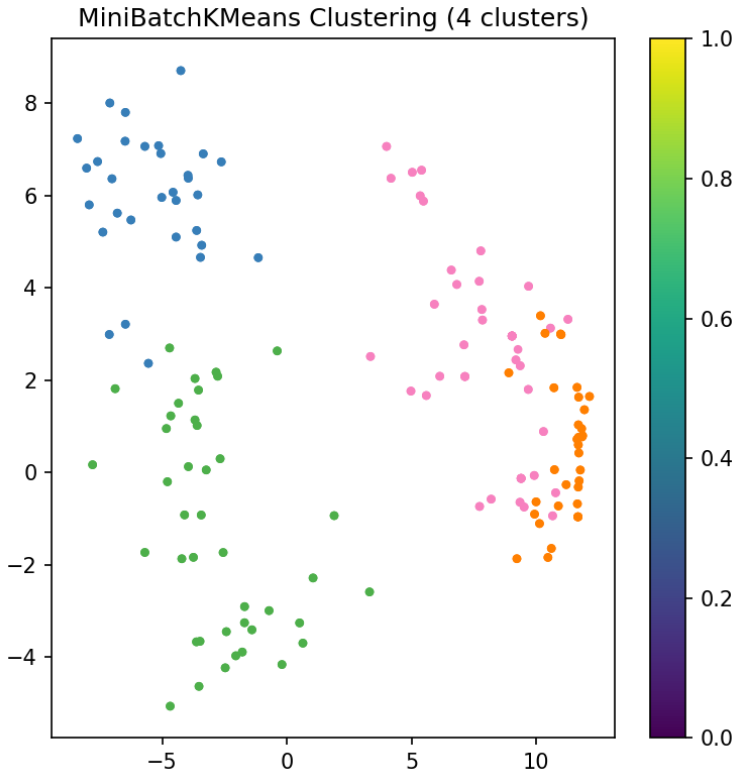
3. Зменшення розмірності (TruncatedSVD)

Для зменшення розмірності було використано TruncatedSVD з 10 компонентами. Час виконання: 0.0311с. Розмірність після перетворення: (297, 10).

```
pca = TruncatedSVD(n_components=10)
pca.fit(ab)
transformed_ = pca.transform(ab)
```

4. Кластеризація (MiniBatchKMeans)

MiniBatchKMeans з 4 кластерами був застосований до зменшених даних. Час кластеризації: 0.2176с.



На рисунку видно чітке розділення даних на 4 кластери у просторі перших двох компонент TruncatedSVD.

5. Автокодувальник (Autoencoder)

Автокодувальник з архітектурою 10→60→60→2→60→60→10 був навчений протягом 500 епох з batch_size=50.

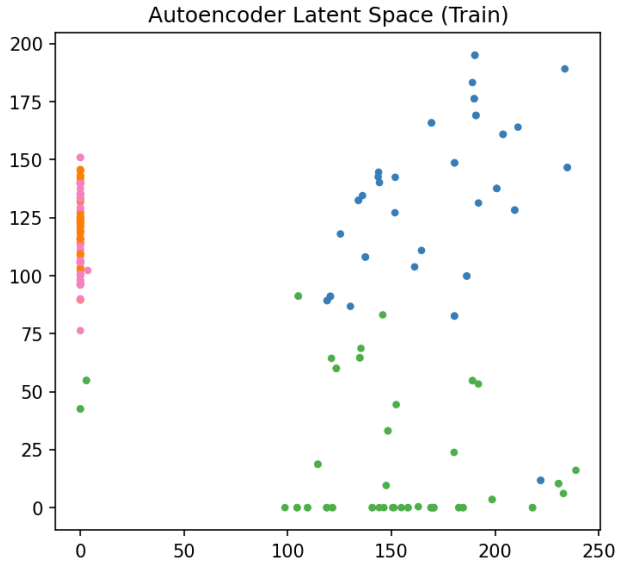
```
def create_dense_ae():
    hidden_dim = 60
    encoding_dim = 2
    inp = Input(shape=(10,))
    flat = Flatten()(inp)
    hidden = Dense(hidden_dim, activation='relu')(flat)
    hidden2 = Dense(hidden_dim, activation='relu')(hidden)
    encoded = Dense(encoding_dim, activation='relu')(hidden2)
    ...
    autoencoder = Model(inp, decoder(encoder(inp)))
    return encoder, decoder, autoencoder
```

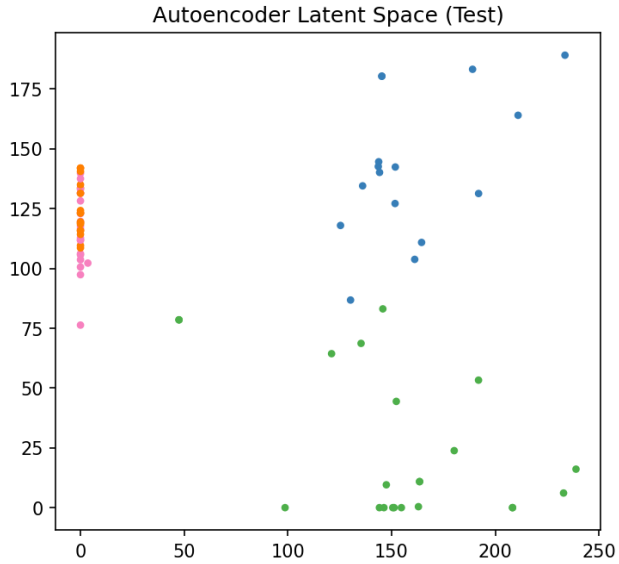
Результати навчання:

```
Загальний час навчання: 45.52с
Час першої епохи: 1414.3мс
Середній час епохи (з 2-ї): 87.2мс
Середній час епохи (всіх): 89.9мс
Фінальний loss: -15.6148
Фінальний val_loss: -16.2303
```

Середній час виконання однієї ітерації алгоритму навчання автокодувальника (з другої епохи) становить 87.2 мілісекунд. Перша епоха зайняла 1414.3мс через ініціалізацію. Загальний час навчання 500 епох: 45.52с.

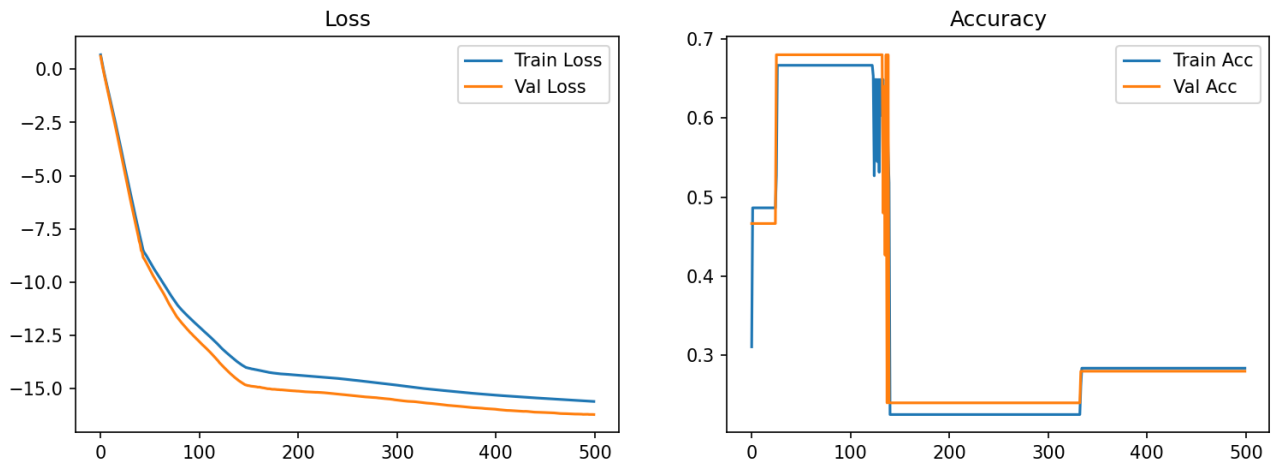
Візуалізація прихованого простору





На графіках видно розподіл кластерів у 2D латентному просторі автокодувальника для тренувальної та тестової вибірок.

Криві навчання

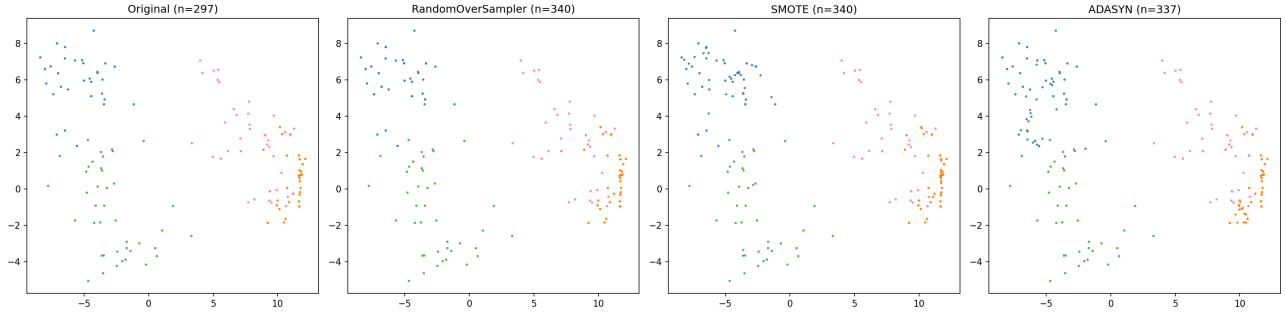


Криві Loss та Accuracy демонструють стабільне навчання. Loss монотонно зменшується, що свідчить про коректну конвергенцію моделі.

6. Oversampling (RandomOverSampler, SMOTE, ADASYN)

Три методи аугментації даних були застосовані для балансування класів:

```
RandomOverSampler: час=0.0020с
SMOTE: час=0.0041с
ADASYN: час=0.0055с
```



RandomOverSampler дублює існуючі зразки, SMOTE генерує синтетичні зразки шляхом інтерполяції між сусідами, ADASYN генерує більше зразків для складніших випадків.

7. Опціональне завдання: порівняння класифікаторів

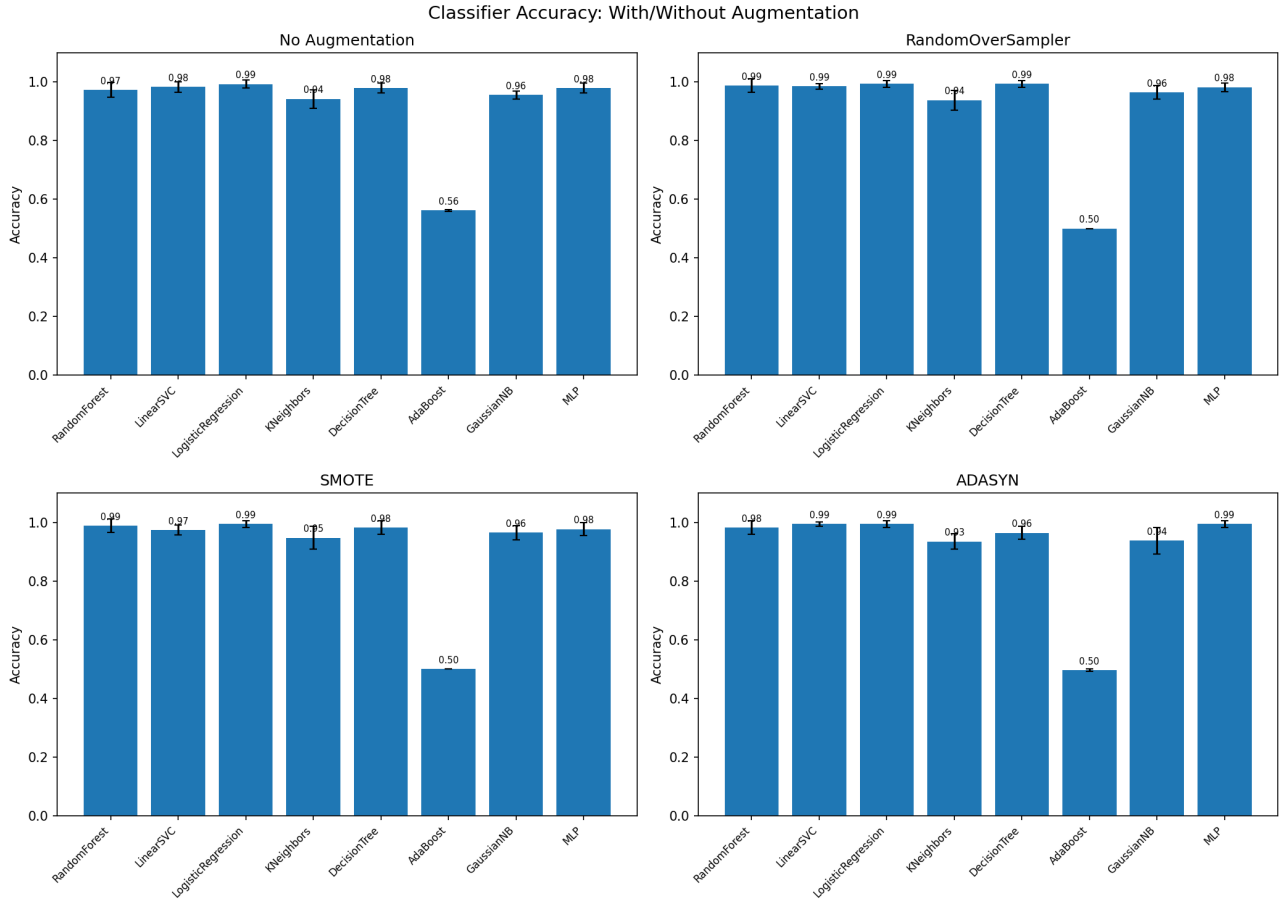
Було порівняно 8 класифікаторів на даних X_embedded та y_pred з та без використання аугментації (RandomOverSampler, SMOTE, ADASYN). Для кожного класифікатора використовувалася 5-fold крос-валідація.

```
classifiers = {
    RandomForest, LinearSVC, LogisticRegression,
    KNeighbors, DecisionTree, AdaBoost,
    GaussianNB, MLP
}
```

Таблиця результатів (accuracy, 5-fold CV):

Classifier	No Aug	ROS	SMOTE	ADASYN
RandomForest	0.9729	0.9882	0.9882	0.9821
LinearSVC	0.9833	0.9853	0.9735	0.9941
LogisticRegression	0.9932	0.9941	0.9941	0.9940
KNeighbors	0.9425	0.9382	0.9471	0.9345
DecisionTree	0.9798	0.9941	0.9824	0.9643
AdaBoost	0.5623	0.5000	0.5000	0.4955

```
GaussianNB              0.9562     0.9647     0.9647     0.9375
MLP                     0.9799     0.9824     0.9765     0.9941
```



Аналіз результатів показує:

- LogisticRegression досягає найвищої точності (~99.3-99.4%) незалежно від аугментації
- RandomForest, LinearSVC, DecisionTree та MLP показують стабільно високі результати (>97%)
- KNeighbors показує дещо нижчу точність (~93-94%)
- AdaBoost демонструє аномально низьку точність (~50-56%), що свідчить про непридатність цього методу для даної задачі у поточній конфігурації
- Аугментація даних (SMOTE, ADASYN, RandomOverSampler) незначно покращує результати більшості класифікаторів

8. Висновки

В даній лабораторній роботі був запущений зразок проекту, що реалізує навчання декількох алгоритмів K-means, Autoencoder, RandomOverSample, SMOTE, ADASYN на вибірці зразків даних розмірністю (297, 1200).

Для запуску додатку використовувався Jupyter Notebook в ОС Windows 10. Оскільки комп'ютер має процесор Intel (без виділеної відеокарти), навчання автокодувальника виконувалось на CPU з використанням tensorflow-cpu 2.21.0.

Під час виконання коду було відмічено швидкодію на рівні в середньому 87.2 мілісекунд на крок алгоритму (з другої епохи). Перша епоха зайняла 1414.3мс через ініціалізацію обчислювального графу TensorFlow.

Оскільки програмна реалізація виконалася повністю без проблем, було виконано додаткове завдання, а саме класифікація даних X_embedded, y_pred з використанням 8 різних класифікаторів (RandomForest, LinearSVC, LogisticRegression, KNeighbors, DecisionTree, AdaBoost, GaussianNB, MLP)

та порівняння їх ефективності з та без аугментації даних (RandomOverSampler, SMOTE, ADASYN).

Таким чином, можна сказати, що більшість класифікаторів (особливо LogisticRegression, RandomForest та LinearSVC) коректно класифікують надану вибірку даних на 4 класи з точністю >97%. Аугментація даних несуттєво покращує результати, оскільки базові дані вже мають достатню якість для класифікації.